

# Day 5 实战项目：生产级部署方案

## \*\* 项目结构

```
deployment/
+-- .github/
|   +-- workflows/
|       +-- ci-cd.yml           # GitHub Actions CI/CD流水线
+-- docker-compose.prod.yml    # 生产环境Docker Compose配置
+-- nginx/
|   +-- prod.conf              # 生产环境Nginx配置
|   +-- ssl/                   # SSL证书目录
|       +-- README.md         # 证书说明
+-- monitoring/
|   +-- prometheus.yml         # 监控配置
+-- README.md                  # 部署文档
```

## \*>> 快速开始

### ☐1. 本地测试（开发环境）

```
# 使用Day 1的starter-project作为示例应用
cd ../../01-开发基础与环境配置/starter-project/docker
docker-compose up -d --build

# 访问 http://localhost:3000
```

### ☐2. 生产环境部署（使用本项目）

```
# 1. 配置环境变量
cp .env.example .env

# 编辑 .env 文件，填入真实配置

# 2. 启动生产服务
docker-compose -f docker-compose.prod.yml up -d --build

# 3. 查看日志
docker-compose -f docker-compose.prod.yml logs -f

# 4. 健康检查
curl http://localhost/health
```

### ☐ 3. 配置CI/CD自动部署

- ☐ Fork或Clone代码到GitHub仓库
- ☐ 在GitHub仓库中启用Actions
- ☐ 推送代码后自动触发构建和部署

## ✓ 功能清单

### ☐ CI/CD流水线 (GitHub Actions)

- ☐ [x] 代码质量检查 (Lint)
- ☐ [x] 单元测试运行
- ☐ [x] Docker镜像构建
- ☐ [x] 镜像推送到容器注册表
- ☐ [x] 自动部署到服务器
- ☐ [x] 通知机制 (成功/失败)

### ☐ Docker生产配置

- ☐ [x] 多阶段构建优化 (减小镜像体积)
- ☐ [x] 安全加固 (非root用户运行)
- ☐ [x] 资源限制 (CPU/内存)
- ☐ [x] 健康检查和自动重启
- ☐ [x] 日志管理 (轮转策略)
- ☐ [x] 网络隔离

### ☐ Nginx反向代理

- ☐ [x] HTTP/HTTPS支持
- ☐ [x] SSL/TLS加密 (Let's Encrypt)
- ☐ [x] Gzip压缩
- ☐ [x] 静态资源缓存
- ☐ [x] 反向代理到后端API
- ☐ [x] 负载均衡 (可选)
- ☐ [x] 访问日志

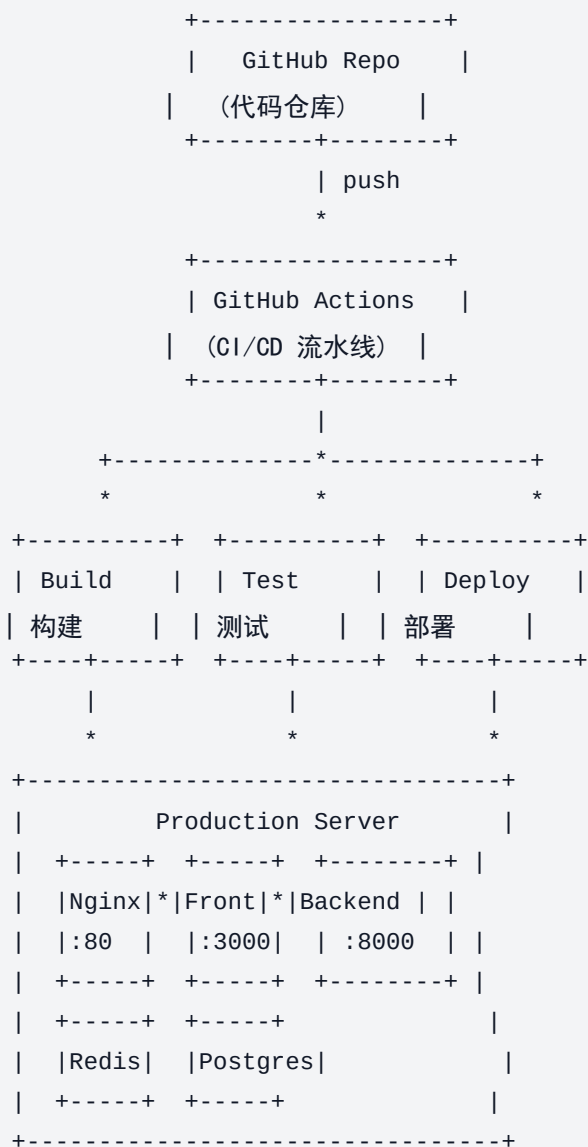
## ☐监控系统

- ☐ [x] Prometheus指标收集
- ☐ [x] 应用性能监控
- ☐ [x] 容器状态监控

---

## \*\* 架构图

---



## \*📄相关文档

本项目对应 Day 5 的以下学习内容：

- ☐ Docker容器编排和生产优化
- ☐ CI/CD自动化流程设计
- ☐ Nginx反向代理配置
- ☐ HTTPS安全证书部署
- ☐ 监控和日志系统

下一步：Day 6：AI基础理论与国内大模型应用